

天津大学
智能与计算学部
网络空间安全学院

《软件安全》实验报告

实验二：基于 UAF 漏洞泄漏 glibc 基地址实验

姓名：b1qx

一. 实验环境

64 位 Ubuntu 16.04 操作系统, glibc 2.23.

二. 实验目的

在实验环境下, 泄漏 glibc 装载基地址是 Pwn 题的关键一步。本实验基于给定的 uaf 漏洞代码, 实现泄漏 glibc 基地址目的。

1. 掌握 glibc 堆管理结构;
2. 会计算 main_arena 地址;
3. 会查找 glibc 基地址与 main_arena 的固定偏移。

三. 实验步骤

(一) 实验原理

Use After Free (UAF, 释放后使用) 是指程序释放一块动态内存后, 仍然通过原指针访问该内存。调用 `free(p)` 后, 指针变量 `p` 本身不会自动变为 `NULL`, 而是继续保存原地址; 此时 `p` 已经成为悬空指针。若程序继续读取该地址, 就可能泄漏堆管理器写入的元数据; 若继续写入, 还可能破坏后来复用该堆块的对象。

本实验在 64 位 Ubuntu 16.04 和 glibc 2.23 环境中申请 0x80 字节内存。加上 chunk 头部后, 其实际 chunk 大小通常为 0x90。该大小不属于本实验环境中的 fast bin 范围, 释放后可进入 unsorted bin。glibc 会在已释放 chunk 的用户数据区域开头写入 fd 和 bk 指针, 因此通过已释放的 `p` 读取前 8 字节, 可以获得一个指向 `main_arena` 附近的地址。

在 64 位 glibc 2.23 中, unsorted bin 尾节点的 fd 指针指向 `main_arena+88`, 即 `main_arena=fd-0x58`。随后通过分析当前系统实际使用的 `libc.so.6`, 得到 `__malloc_hook` 在 `libc` 中的偏移为 0x3c4b10; 由于 `main_arena` 位于 `__malloc_hook` 之后 0x10 字节, 因此 `main_arena` 相对 `libc` 基地址的固定偏移为 0x3c4b20。最终可以利用“`libc 基地址=main_arena-0x3c4b20`”完成地址泄漏。

(二) 实验环境与工具

实验使用 64 位 Ubuntu 16.04 操作系统, glibc 版本为 2.23。主要工具包括 GCC 编译器、GDB 调试器、nm 符号查看工具以及 Python 3。虚拟机中已安装 open-vm-tools, 便于在宿主机和虚拟机之间复制代码及实验截图。

```
uname -m
getconf LONG_BIT
ldd --version | head -n 1
gcc -g -O0 -Wall uaf.c -o uaf
file ./uaf
```

(三) 编写 UAF 漏洞程序

在主函数中使用 `malloc(0x80)` 申请堆内存, 打印其地址后调用 `free(p)` 释放。随后故意通过已经释放的 `p` 读取用户区前 8 字节, 并将其解释为 `uintptr_t` 类型的 fd 指针。程序再依次计算 `main_arena` 地址和 `libc` 装载基地址。核心代码如下。

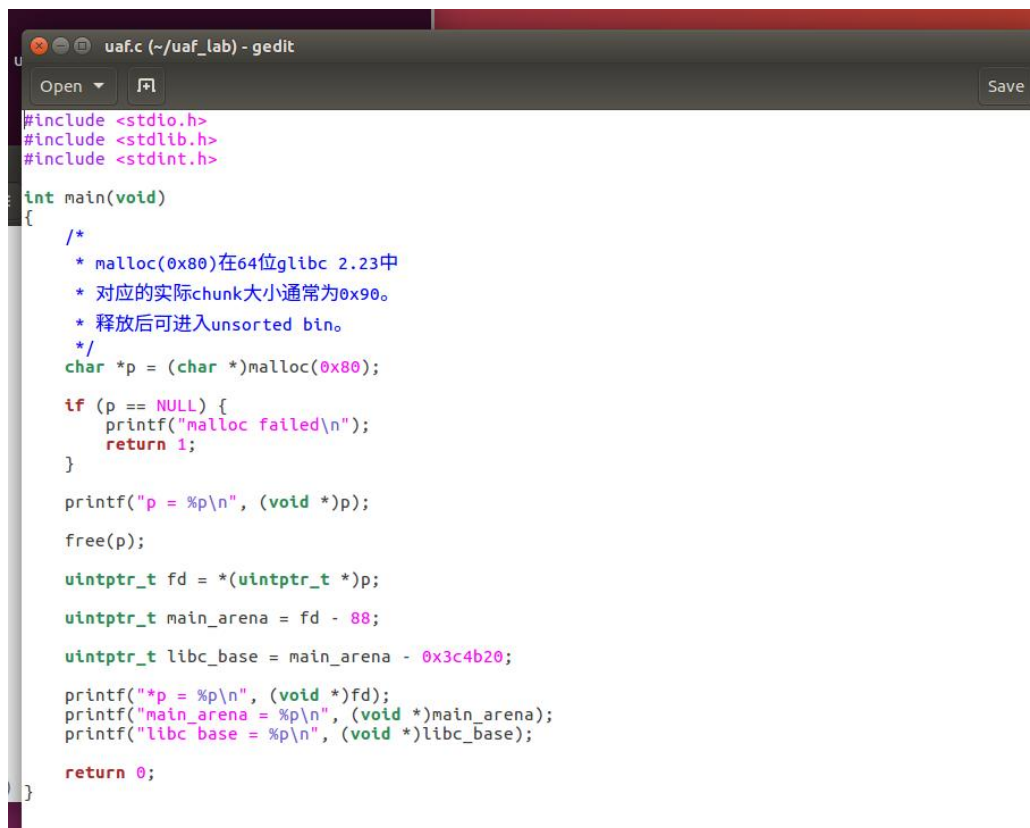
```

char *p = (char *)malloc(0x80);
printf("p = %p\n", (void *)p);
free(p);

/* UAF: 释放后仍通过 p 读取 chunk 用户区 */
uintptr_t fd = *(uintptr_t *)p;
uintptr_t main_arena = fd - 0x58;
uintptr_t libc_base = main_arena - 0x3c4b20;

printf("*p = %p\n", (void *)fd);
printf("main_arena = %p\n", (void *)main_arena);
printf("libc base = %p\n", (void *)libc_base);

```



```

uaf.c (~/.uaf_lab) - gedit
Open Save
#include <stdio.h>
#include <stdlib.h>
#include <stdint.h>

int main(void)
{
    /*
     * malloc(0x80)在64位glibc 2.23中
     * 对应的实际chunk大小通常为0x90。
     * 释放后可进入unsorted bin。
     */
    char *p = (char *)malloc(0x80);

    if (p == NULL) {
        printf("malloc failed\n");
        return 1;
    }

    printf("p = %p\n", (void *)p);
    free(p);

    uintptr_t fd = *(uintptr_t *)p;
    uintptr_t main_arena = fd - 88;
    uintptr_t libc_base = main_arena - 0x3c4b20;

    printf("*p = %p\n", (void *)fd);
    printf("main_arena = %p\n", (void *)main_arena);
    printf("libc base = %p\n", (void *)libc_base);

    return 0;
}

```

图 1 UAF 实验源代码

(四) 编译并直接运行程序

使用-g 保留调试信息，使用-O0 关闭编译优化，使用-Wall 显示常见警告。由于实验代码有意在 free 后继续读取 p，编译器可能提示 use-after-free 相关警告，这是本实验主动构造漏洞造成的。运行程序后，再使用 file 命令确认生成文件为 64 位 ELF 可执行文件。

```

gcc -g -O0 -Wall uaf.c -o uaf
./uaf
file ./uaf

```

```
b1qx@ubuntu:~$ vmware-toolbox-cmd -v
10.2.0.1608 (build-7253323)
b1qx@ubuntu:~$ mkdir -p ~/uaf_lab
b1qx@ubuntu:~$ cd ~/uaf_lab
b1qx@ubuntu:~/uaf_lab$ nano uaf.c
b1qx@ubuntu:~/uaf_lab$ gcc -g -O0 -Wall uaf.c -o uaf
b1qx@ubuntu:~/uaf_lab$ ./uaf
p = 0x1c7f010
*p = 0x7fa672e94b78
main_arena = 0x7fa672e94b20
libc base = 0x7fa672ad0000
b1qx@ubuntu:~/uaf_lab$ file ./uaf
./uaf: ELF 64-bit LSB executable, x86-64, version 1 (SYSV), dynamically linked,
interpreter /lib64/ld-linux-x86-64.so.2, for GNU/Linux 2.6.32, BuildID[sha1]=768
7357d6c5dd991608dab0adeaf25c1e930cc34, not stripped
b1qx@ubuntu:~/uaf_lab$
```

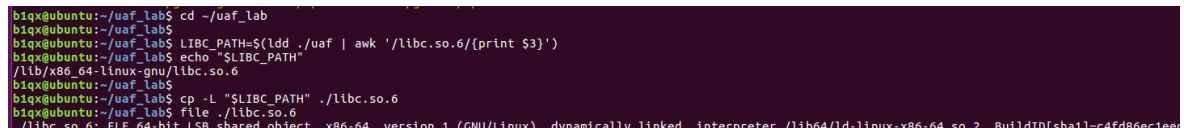
图2 程序编译、运行结果及 64 位 ELF 文件确认

本次直接运行得到 $p=0x1c7f010$ ，释放后读取到 $fd=0x7fa672e94b78$ 。根据 64 位环境的固定关系， $main_arena=0x7fa672e94b78-0x58=0x7fa672e94b20$ ；再减去 $main_arena$ 相对 $libc$ 的偏移 $0x3c4b20$ ，得到 $libc$ 基地址 $0x7fa672ad0000$ 。结果为页对齐地址，符合共享库装载基址的一般特征。

（五）使用方法二确定 main_arena 固定偏移

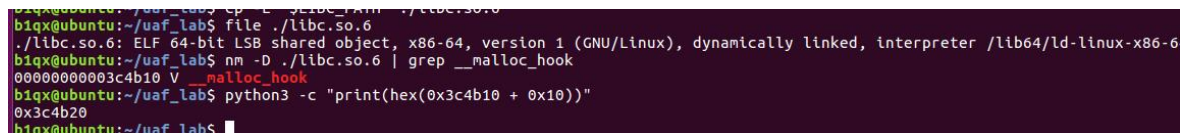
为避免直接照搬课件中的常数，本实验按照课件“方法二”分析当前程序实际加载的 libc.so.6。首先通过 ldd 取得 libc 路径并复制到实验目录，再使用 nm -D 读取动态符号表中的 __malloc_hook。

```
LIBC_PATH=$(ldd ./uaf | awk '/libc.so.6/{print $3}')
cp -L "$LIBC_PATH" ./libc.so.6
file ./libc.so.6
nm -D ./libc.so.6 | grep __malloc_hook
```



```
biqx@ubuntu:~/uaf_lab$ cd ~/uaf_lab
biqx@ubuntu:~/uaf_lab$ LIBC_PATH=$(ldd ./uaf | awk '/libc.so.6/{print $3}')
biqx@ubuntu:~/uaf_lab$ echo "$LIBC_PATH"
/lib/x86_64-linux-gnu/libc.so.6
biqx@ubuntu:~/uaf_lab$ cp -L "$LIBC_PATH" ./libc.so.6
biqx@ubuntu:~/uaf_lab$ file ./libc.so.6
./libc.so.6: ELF 64-bit LSB shared object, x86_64, version 1 (GNU/Linux), dynamically linked, interpreter /lib64/ld-linux-x86-64.so.2, BuildID[sha1]=c4d86ec1ea
```

图3 定位并复制程序实际使用的 libc.so.6



```
biqx@ubuntu:~/uaf_lab$ cp -L "$LIBC_PATH" ./libc.so.6
biqx@ubuntu:~/uaf_lab$ file ./libc.so.6
./libc.so.6: ELF 64-bit LSB shared object, x86_64, version 1 (GNU/Linux), dynamically linked, interpreter /lib64/ld-linux-x86-6
biqx@ubuntu:~/uaf_lab$ nm -D ./libc.so.6 | grep __malloc_hook
0000000003c4b10 V __malloc_hook
biqx@ubuntu:~/uaf_lab$ python3 -c "print(hex(0x3c4b10 + 0x10))"
0x3c4b20
biqx@ubuntu:~/uaf_lab$
```

图4 查找 __malloc_hook 并计算 main_arena 偏移

nm 输出表明 __malloc_hook 的偏移为 0x3c4b10。根据 glibc 2.23 中二者的布局关系，main_arena 偏移为 __malloc_hook+0x10，因此计算结果为 0x3c4b20。实验中使用 Python 3 执行十六进制加法，输出与课件一致。

```
python3 -c "print(hex(0x3c4b10 + 0x10))"
```

因此，本实验使用的两级地址换算关系为： $\text{main_arena} = \text{fd} - 0x58$ ； $\text{libc_base} = \text{main_arena} - 0x3c4b20$ 。该固定偏移只适用于本实验所分析的 glibc 2.23 文件，换用其他 glibc 版本时应重新读取符号并确认内部结构。

（六）使用 GDB 观察 free 前后的堆内存

启动 GDB 后在 main 函数设置断点，执行 malloc 后查看 p 的值和 p 指向的内存。随后在 free(p)所在行设置断点，在 free 执行前后分别使用 x/6gx p 查看 6 个 8 字节数据。主要调试命令如下。

```
gdb ./uaf
set pagination off
set disassembly-flavor intel
break main
run
next
p/x p
x/6gx p
break uaf.c:21
continue
next
x/6gx p
```

```

or help, type help.
type "apropos word" to search for commands related to "word"...
Reading symbols from ./uaf...done.
(gdb) set pagination off
(gdb) set disassembly-flavor intel
(gdb) break main
breakpoint 1 at 0x4005fe: file uaf.c, line 12.
(gdb) run
Starting program: /home/biqx/uaf_lab/uaf

breakpoint 1, main () at uaf.c:12
12      char *p = (char *)malloc(0x80);
(gdb) list
12      /*
13      * malloc(0x80)在64位glibc 2.23中
14      * 对应的实际chunk大小通常为0x90。
15      * 释放后可进入unsorted bin。
16      */
17      char *p = (char *)malloc(0x80);
18
19      if (p == NULL) {
20          printf("malloc failed\n");
21          return 1;
22      }
23
24      printf("p = %p\n", (void *)p);
25
26      free(p);
27
28      uintptr_t fd = *(uintptr_t *)p;
29
30      uintptr_t main_arena = fd - 88;
31
32      uintptr_t libc_base = main_arena - 0x3c4b20;
33
34      printf("*p = %p\n", (void *)fd);
35      printf("main_arena = %p\n", (void *)main_arena);
36      printf("libc base = %p\n", (void *)libc_base);
37
38      return 0;
39  }
(gdb)
line number 35 out of range; uaf.c has 34 lines.
(gdb) next
14      if (p == NULL) {
(gdb) p/x p
p1 = 0x602010
(gdb) x/6gx
Argument required (starting display address).
(gdb) x/6gx p
0x602010:      0x0000000000000000      0x0000000000000000
0x602020:      0x0000000000000000      0x0000000000000000
0x602030:      0x0000000000000000      0x0000000000000000
(gdb) break uaf.c:21
breakpoint 2 at 0x40063d: file uaf.c, line 21.
(gdb) continue
Continuing.
p = 0x602010

breakpoint 2, main () at uaf.c:21
21      free(p);
(gdb) p/x p
p2 = 0x602010
(gdb) x/6gx p
0x602010:      0x0000000000000000      0x0000000000000000
0x602020:      0x0000000000000000      0x0000000000000000
0x602030:      0x0000000000000000      0x0000000000000000
(gdb)

```

图5 GDB 设置断点并查看 free 执行前的堆内存

在 free 执行前，p=0x602010，x/6gx p 显示用户数据区的内容均为 0。此时该 chunk 仍然处于已分配状态，用户区域尚未被堆管理器用作空闲链表节点。截图中第一次执行 x/6gx 时因未给出起始地址而提示“Argument required”，补充参数 p 后即可正常显示，不影响实验结果。


```

(gdb) next
23      uintptr_t fd = *(uintptr_t *)p;
(gdb) p/x p
$3 = 0x602010
(gdb) x/6gx p
0x602010:      0x00007ffff7dd1b78      0x00007ffff7dd1b78
0x602020:      0x0000000000000000      0x0000000000000000
0x602030:      0x0000000000000000      0x0000000000000000
(gdb)

```

图 6 free 执行后用户区开头被写入 fd 和 bk 指针

执行 free(p)后，指针变量 p 仍为 0x602010，但 p 已经不再拥有该内存。再次查看内存可见前两个 8 字节均变为 0x7ffff7dd1b78，分别对应 unsorted bin 空闲 chunk 中的 fd 和 bk。通过已经释放的 p 读取该值就是本实验的 UAF 信息泄漏点。

(七) 地址计算与结果分析

根据 GDB 中泄漏的 fd=0x7ffff7dd1b78，可得到：

```

main_arena = 0x7ffff7dd1b78 - 0x58 = 0x7ffff7dd1b20
libc_base  = 0x7ffff7dd1b20 - 0x3c4b20 = 0x7ffff7a0d000

```

GDB 调试时的地址与直接运行时不同，是因为系统启用了 ASLR，每次启动进程时堆和共享库的装载地址都会随机变化。但是 fd 与 main_arena 之间的 0x58 偏移，以及 main_arena 相对同一 libc 文件基地址的 0x3c4b20 偏移保持不变。因此，攻击者即使面对 ASLR，也可以利用 UAF 泄漏值恢复当前进程的 libc 基地址。

综合程序直接运行和 GDB 调试结果，可以确认：程序成功产生悬空指针；free 后的 chunk 进入 unsorted bin；用户区开头保存了指向 main_arena 附近的 fd 和 bk；按照固定偏移能够正确计算 libc 装载基地址。本实验达到了基于 UAF 漏洞泄漏 glibc 基地址的目的。

(八) 实验中遇到的问题及解决方法

1. 初次环境为 32 位 Ubuntu，file 命令显示 ELF 32-bit，而课件代码使用 64 位偏移 88。32 位环境中 unsorted bin 相关偏移不同，无法直接复现课件结果。重新安装 Ubuntu 16.04 amd64 后，uname -m 显示 x86_64，getconf LONG_BIT 显示 64，问题解决。
2. 安装 Pwntools 时，Ubuntu 16.04 自带的 Python 版本与新版依赖不兼容，出现

setuptools 和 psutil 语法错误。由于本实验只需读取 `__malloc_hook` 符号，最终使用 `nm -D` 直接查看 `libc` 动态符号表，并使用 Python 3 完成 0x10 加法，获得与 Pwntools 方法等价的结果。

3. 在 GDB 中第一次输入 `x/6gx` 时遗漏了起始地址，GDB 提示需要参数。将命令改为 `x/6gx p` 后即可正确显示 `p` 指向的内存。

四. 心得体会

通过本次实验，我对 UAF 漏洞的形成原因和 `glibc` 堆管理结构有了更直观的认识。`free` 函数只改变堆块的管理状态，并不会清除所有旧指针；因此程序员如果继续使用原指针，就可能读取到堆管理器写入的 `fd`、`bk` 等元数据。由此可见，UAF 不仅会造成程序崩溃或数据错误，还可能成为地址泄漏和控制流劫持的基础。

实验中通过 GDB 对比 `free` 前后的内存，清楚观察到用户数据区被转换为空闲链表节点的过程。通过 `__malloc_hook` 符号计算 `main_arena` 固定偏移，也使我理解了“泄漏一个 `libc` 内部地址，再通过固定偏移恢复 `libc` 基址”的基本思路。ASLR 会改变每次运行的绝对地址，但无法改变同一模块内部的相对偏移，因此信息泄漏能够削弱地址随机化保护。

实验还说明，环境位数和 `glibc` 版本会直接影响偏移，漏洞复现必须以实际二进制为准，并在安全编码中避免继续使用已释放指针。